



**SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL – SENAI
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS – NOITE**

OTAVIO HENRIQUE BARBOSA SOARES

**DOCUMENTAÇÃO INDIVIDUAL DE TESTES UNITÁRIOS
MÓDULO MAPA E PONTOS DE COLETA – SISTEMA REGRAPHIK**

NOVA LIMA

2026

OTAVIO HENRIQUE BARBOSA SOARES

DOCUMENTAÇÃO INDIVIDUAL DE TESTES UNITÁRIOS
MÓDULO MAPA E PONTOS DE COLETA – SISTEMA REGRAPHIK

Documentação individual apresentada como parte da atividade de testes unitários aplicada ao Projeto de TCC do curso Técnico em Desenvolvimento de Sistemas – Noite, sob orientação do Instrutor Frederico Martins Aguiar.

NOVA LIMA

2026

SUMÁRIO

1 INTRODUÇÃO

2 IDENTIFICAÇÃO DA ATIVIDADE

3 ORGANIZAÇÃO E METODOLOGIA DA SUÍTE

4 ADAPTAÇÕES NECESSÁRIAS NO CÓDIGO ORIGINAL

4.1 GooglePlacesService

4.2 PontosColetaService

4.3 Componentes sem alteração

4.4 Projeto TesteReGraphik

5 PLANO DE TESTES UNITÁRIOS – WEB

5.1 WEB / ConvertersTeste

5.2 WEB / ServicesTeste

5.3 WEB / ViewModelsTeste

6 PLANO DE TESTES UNITÁRIOS – API

6.1 APITeste / ControllerTeste

6.2 APITeste / ServicesApiTeste

7 REGISTRO DOS RESULTADOS DA SUÍTE

8 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO

Este documento apresenta o planejamento dos testes unitários do módulo de Mapa e Pontos de Coleta do sistema ReGraphik. A organização foi preparada como roteiro de execução e de evidências, sem registrar previamente qualquer teste como concluído. Cada caso de teste contém objetivo, procedimento previsto, resultado esperado e um espaço próprio para inserção do print do código correspondente.

A suíte será implementada com xUnit e seguirá a estrutura Arrange, Act e Assert. Os cenários foram definidos a partir dos comportamentos das classes relacionadas ao mapa, considerando caminhos válidos, entradas inesperadas, exceções, respostas de serviços externos e regras de negócio que podem ser verificadas de forma isolada.

Além do plano de testes, o documento registra as adaptações de testabilidade que foram aplicadas ao código original para permitir o isolamento de Google Places e Firebase. Essas adaptações não têm como objetivo alterar regras de negócio; elas permitem que os testes controlem dependências externas e sejam executados de maneira repetível.

2 IDENTIFICAÇÃO DA ATIVIDADE

Aluno	Otávio Henrique Barbosa Soares
Turma	Técnico em Desenvolvimento de Sistemas – Noite
Equipe	ReGraphik
Data da entrega	25/08/2026
Componente	Mapa e Pontos de Coleta
Projeto de testes	TesteReGraphik
Framework de testes	xUnit
Ambiente	C# / .NET 8 / Visual Studio 2022

2.1 Escopo individual

O escopo desta documentação será limitado às classes relacionadas ao mapa e aos pontos de coleta. Os testes serão organizados nas pastas WEB/ConvertersTeste, WEB/ServicesTeste, WEB/ViewModelsTeste, APITeste/ControllerTeste e APITeste/ServicesApiTeste. Alterações em componentes de outras funcionalidades da equipe não fazem parte deste trabalho.

3 ORGANIZAÇÃO E METODOLOGIA DA SUÍTE

A suíte será composta por 46 casos de teste, identificados de CT001 a CT046. Cada caso deverá ser implementado em sua classe correspondente e documentado individualmente. Após a execução de cada pasta, será inserido também um print geral do Gerenciador de Testes mostrando o conjunto daquela pasta.

3.1 Estrutura planejada



3.2 Padrão Arrange, Act e Assert

Cada método de teste será organizado em três etapas. Arrange preparará os objetos e dados necessários; Act executará o comportamento que está sendo avaliado; Assert comparará o resultado obtido com o comportamento esperado. Os nomes dos métodos permanecerão em português sempre que possível, preservando apenas termos técnicos do framework ou da própria aplicação.

3.3 Padrão de evidências

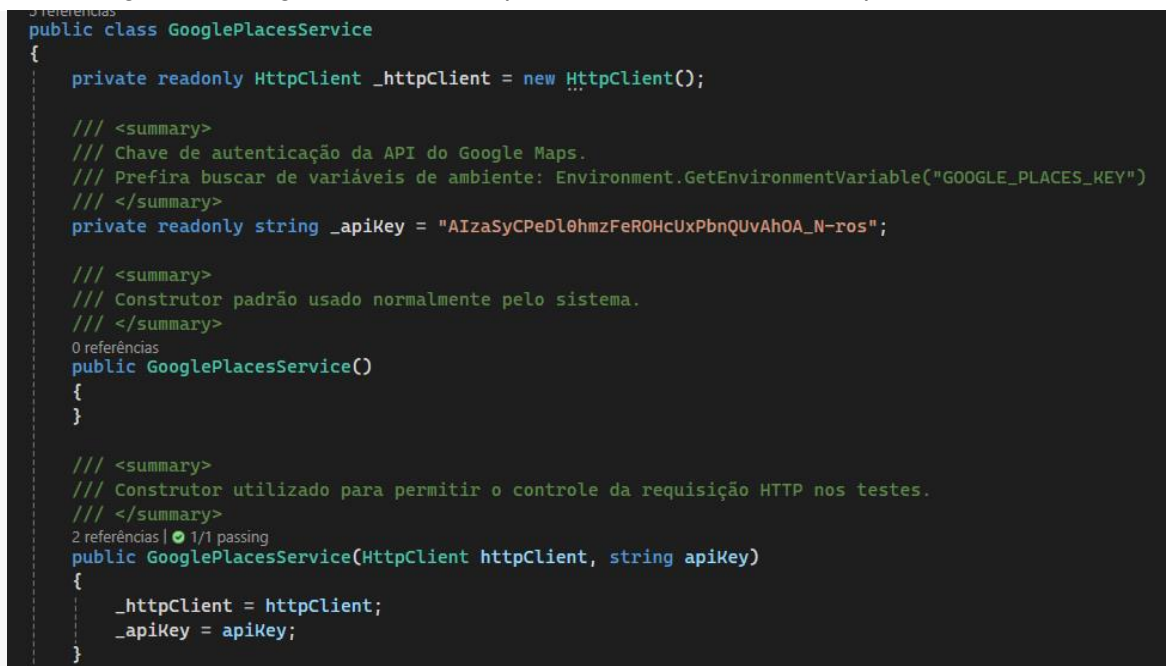
Para cada CT haverá um espaço reservado para o print do método de teste. O print deverá mostrar, sempre que possível, o atributo [Fact], o nome do método e as etapas Arrange, Act e Assert. Ao final de cada pasta haverá um espaço adicional para o print geral do Gerenciador de Testes. O campo “Resultado após execução” deverá ser preenchido somente depois que o teste for realmente executado.

4.1.2 Estrutura a ser adicionada

```
public GooglePlacesService()
{
}

public GooglePlacesService(HttpClient httpClient, string apiKey)
{
    _httpClient = httpClient;
    _apiKey = apiKey;
}
```

Figura 2 – GooglePlacesService após inclusão dos construtores para testabilidade



```
public class GooglePlacesService
{
    private readonly HttpClient _httpClient = new HttpClient();

    /// <summary>
    /// Chave de autenticação da API do Google Maps.
    /// Prefira buscar de variáveis de ambiente: Environment.GetEnvironmentVariable("GOOGLE_PLACES_KEY")
    /// </summary>
    private readonly string _apiKey = "AIzaSyCPeDl0hmzFeROHcUxPbnQUvAhOA_N-ros";

    /// <summary>
    /// Construtor padrão usado normalmente pelo sistema.
    /// </summary>
    0 referências
    public GooglePlacesService()
    {
    }

    /// <summary>
    /// Construtor utilizado para permitir o controle da requisição HTTP nos testes.
    /// </summary>
    2 referências | 1/1 passing
    public GooglePlacesService(HttpClient httpClient, string apiKey)
    {
        _httpClient = httpClient;
        _apiKey = apiKey;
    }
}
```

Fonte: Elaborado pelo autor (2026).

Testes relacionados: CT006 a CT012. A finalidade dessa adaptação será permitir que o arquivo GooglePlacesServiceTeste.cs utilize um HttpClient controlado, sem realizar chamadas reais à internet.

4.2 PontosColetaService

Situação original: A classe possuía apenas o construtor público responsável pela inicialização normal do Firebase.

Adaptação realizada: O construtor público original foi preservado. Foi acrescentado um segundo construtor, com modificador `protected`, que recebe uma instância de `FirebaseClient` fornecida externamente.

Motivo: Permitir que uma classe derivada utilizada nos testes seja criada com um cliente Firebase controlado, sem executar o processo real de leitura de credenciais e autenticação.

Impacto: O funcionamento do construtor utilizado em produção não foi alterado.

4.2.1 Construtor protegido a ser acrescentado

```
protected PontosColetaService(
    ILogger<PontosColetaService> logger,
    IConfiguration configuration,
    FirebaseClient firebaseClient)
{
    _logger = logger;
    _configuration = configuration;
    _firebaseClient = firebaseClient;
}
```

Figura 3 – Estrutura original do PontosColetaService antes da inclusão do construtor para testes

```
        var token = await credenciais.UnderlyingCredential.GetAccessTokenForRequestAsync();
        return token;
    }
}

catch (Exception ex)
{
    _logger.LogCritical($"Falha fatal ao inicializar o FirebaseService: {ex.Message}");
    throw;
}

/// <summary>
/// Lista todos os pontos de coleta cadastrados no ReGraphik, utilizando o repositório para acessar os dados e registrando qualquer erro que
/// possa ocorrer durante a operação.
/// </summary>
/// <returns></returns>
/// <exception cref="Exception">Lançada quando ocorre um erro ao listar os dados</exception>
2 referências
public async Task<List<PontosColeta>> Listar()
{
    try
    {
```

Fonte: Elaborado pelo autor (2026).

Figura 4 – Construtor protegido acrescentado ao PontosColetaService

```

        throw;
    }
}

/// <summary>
/// Construtor alternativo utilizado para permitir o isolamento
/// do Firebase durante a execução dos testes automatizados.
/// </summary>
/// <param name="logger">Logger utilizado pelo serviço.</param>
/// <param name="configuration">Configurações utilizadas pelo serviço.</param>
/// <param name="firebaseClient">Cliente Firebase controlado externamente.</param>
2 referências
protected PontosColetaService(
    ILogger<PontosColetaService> logger,
    IConfiguration configuration,
    FirebaseClient firebaseClient)
{
    _logger = logger;
    _configuration = configuration;
    _firebaseClient = firebaseClient;
}

/// <summary>
/// Lista todos os pontos de coleta cadastrados no ReGraphik.
/// </summary>
4 referências
public virtual async Task<List<PontosColeta>> Listar()

```

Fonte: Elaborado pelo autor (2026).

4.2.2 Métodos que deverão receber o modificador virtual

A lógica interna dos métodos deverá permanecer inalterada. A única mudança será acrescentar virtual às assinaturas abaixo, permitindo override em classes falsas criadas exclusivamente no projeto de testes.

```

public virtual async Task<List<PontosColeta>> Listar()
public virtual async Task<PontosColeta> ObterPorId(string id)
public virtual async Task Criar(PontosColeta pontosColeta)
public virtual async Task<(int salvos, int ignorados)> SincronizarComGoogleMapsAsync(
    string cidade, string apiKey, HttpClient httpClient)
public virtual async Task Atualizar(string id, PontosColeta pontosColeta)
public virtual async Task Excluir(string id)

```

Figura 5 – Assinaturas do PontosColetaService após aplicação de virtual

```

4 referências
public virtual async Task<List<PontosColeta>> Listar()...

/// <summary> Obtém um ponto de coleta específico através do ID.
4 referências
public virtual async Task<PontosColeta> ObterPorId(
    string id)...

/// <summary> Adiciona um novo ponto de coleta ao Firebase.
4 referências
public virtual async Task Criar(
    PontosColeta pontosColeta)...

/// <summary> Sincroniza os pontos de coleta utilizando os resultados encontrado ...
public virtual async Task<(int salvos, int ignorados)>
    SincronizarComGoogleMapsAsync(
        string cidade,
        string apiKey,
        HttpClient httpClient)...

/// <summary> Atualiza um ponto de coleta existente.
2 referências
public virtual async Task Atualizar(
    string id,
    PontosColeta pontosColeta)
{
    try...
}

/// <summary> Exclui um ponto de coleta através do ID.
2 referências
public virtual async Task Excluir(
    string id)

```

Fonte: Elaborado pelo autor (2026).

Testes relacionados: CT027 a CT046. O construtor público original continuará responsável pelo funcionamento normal da aplicação, enquanto o construtor protegido e os métodos virtuais permitirão substituições apenas no contexto dos testes.

4.3 Componentes que não deverão ser alterados

Para os cenários planejados, BoolToVisibilityConverter.cs, MapaViewModel.cs e PontosColetaController.cs não precisarão receber alterações para tornar os testes executáveis. Os testes deverão observar o comportamento já existente nessas classes.

4.4 Projeto TesteReGraphik

O arquivo TesteReGraphik.csproj não deverá ser apresentado como uma alteração realizada nesta documentação, pois ele não foi modificado. A documentação deve refletir somente mudanças efetivamente aplicadas no processo.

4.5 Quadro-resumo das adaptações

Arquivo	Adaptação planejada	Finalidade	CT relacionados
GooglePlacesService.cs	Construtor padrão + construtor com HttpClient e apiKey	Simular respostas do Google Places	CT006–CT012
PontosColetaService.cs	Construtor protected + seis métodos virtual	Isolar Firebase e permitir classes controladas nos testes	CT027–CT046
BoolToVisibilityConverter.cs	Nenhuma	Testar implementação existente	CT001–CT005
MapaViewModel.cs	Nenhuma	Testar implementação existente	CT013–CT026
PontosColetaController.cs	Nenhuma	Testar decisões HTTP da implementação existente	CT027–CT039
TesteReGraphik.csproj	Nenhuma alteração registrada	Refletir o processo realmente realizado	—

5 PLANO DE TESTES UNITÁRIOS – WEB

Esta seção apresentará os casos planejados para a camada WPF. Cada CT terá um espaço próprio para evidência do método de teste. O resultado deverá ser preenchido apenas após a execução.

5.1 WEB / ConvertersTeste

Os casos CT001 a CT005 avaliarão a conversão de valores para estados de visibilidade do WPF.

CT001 – Conversão de valor verdadeiro

Camada / pasta	WEB / ConvertersTeste
Classe de teste	BoolToVisibilityConverterTeste
Nome do método	Converter_ValorVerdadeiro_DeveRetornarVisible
Situação	Aprovado

Objetivo: Verificar se o converter retorna Visibility.Visible quando recebe o valor booleano true.

Procedimento previsto: Será criada uma instância do converter e enviado o valor true ao método Convert. O retorno será comparado com Visibility.Visible.

Resultado esperado: O teste deverá confirmar que o resultado é Visibility.Visible.

Figura 6 – Evidência do código do CT001 – BoolToVisibilityConverterTeste

```

/// <summary> Confere se o valor verdadeiro é convertido para Visible.
[Fact]
0 referências
public void Converter_ValorVerdadeiro_DeveRetornarVisible()
{
    // Arrange
    var converter = new BoolToVisibilityConverter();
    bool valor = true;

    // Act
    var resultado = converter.Convert(
        valor,
        typeof(Visibility),
        null!,
        CultureInfo.InvariantCulture);

    // Assert
    Assert.Equal(Visibility.Visible, resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT002 – Conversão de valor falso

Camada / pasta	WEB / ConvertersTeste
Classe de teste	BoolToVisibilityConverterTeste
Nome do método	Converter_ValorFalso_DeveRetornarCollapsed
Situação	Aprovado

Objetivo: Verificar se o converter retorna Visibility.Collapsed quando recebe o valor booleano false.

Procedimento previsto: Será enviado false ao método Convert, mantendo os demais parâmetros apenas como apoio à chamada do conversor.

Resultado esperado: O teste deverá confirmar que o resultado é Visibility.Collapsed.

Figura 7 – Evidência do código do CT002 – BoolToVisibilityConverterTeste

```

/// <summary> Confere se o valor falso é convertido para Collapsed.
[Fact]
0 referências
public void Converter_ValorFalso_DeveRetornarCollapsed()
{
    // Arrange
    var converter = new BoolToVisibilityConverter();
    bool valor = false;

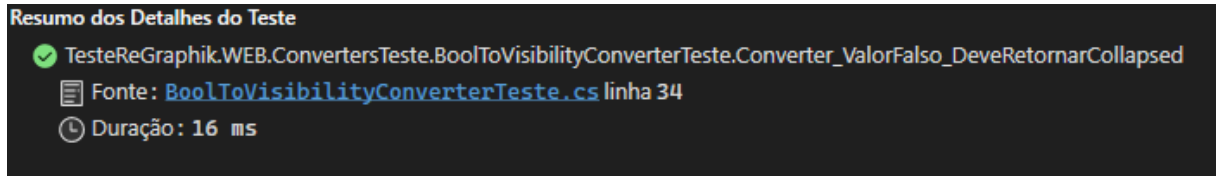
    // Act
    var resultado = converter.Convert(
        valor,
        typeof(Visibility),
        null!,
        CultureInfo.InvariantCulture);

    // Assert
    Assert.Equal(Visibility.Collapsed, resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT003 – Tratamento de valor não booleano

Camada / pasta	WEB / ConvertersTeste
Classe de teste	BoolToVisibilityConverterTeste
Nome do método	Converter_ValorNaoBooleano_DeveRetornarCollapsed
Situação	Aprovado

Objetivo: Verificar o comportamento do converter quando a entrada não é do tipo booleano.

Procedimento previsto: Será utilizada uma string como entrada para representar um valor inesperado.

Resultado esperado: O teste deverá confirmar que a entrada é tratada sem exceção e retorna Visibility.Collapsed.

Figura 8 – Evidência do código do CT003 – BoolToVisibilityConverterTeste

```

/// <summary> Testa o comportamento quando o converter recebe um valor que não é ...
[Fact]
0 referências
public void Converter_ValorNaoBooleano_DeveRetornarCollapsed()
{
    // Arrange
    var converter = new BoolToVisibilityConverter();
    string valor = "true";

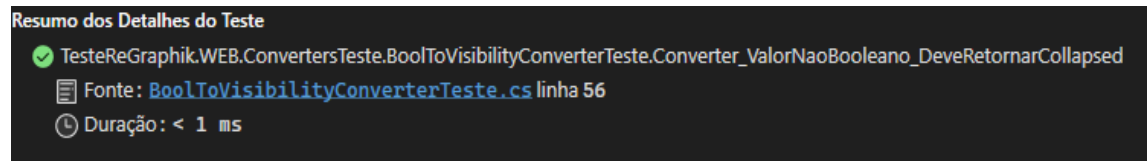
    // Act
    var resultado = converter.Convert(
        valor,
        typeof(Visibility),
        null!,
        CultureInfo.InvariantCulture);

    // Assert
    Assert.Equal(Visibility.Collapsed, resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT004 – Tratamento de valor nulo

Camada / pasta	WEB / ConvertersTeste
Classe de teste	BoolToVisibilityConverterTeste
Nome do método	Converter_ValorNulo_DeveRetornarCollapsed
Situação	Aprovado

Objetivo: Verificar como o converter trata uma entrada nula.

Procedimento previsto: Será enviado um valor nulo ao método Convert.

Resultado esperado: O teste deverá confirmar que o método retorna Visibility.Collapsed.

Figura 9 – Evidência do código do CT004 – BoolToVisibilityConverterTeste

```

/// <summary> Verifica o comportamento do converter quando recebe um valor nulo.
[Fact]
| 0 referências
public void Converter_ValorNulo_DeveRetornarCollapsed()
{
    // Arrange
    var converter = new BoolToVisibilityConverter();
    object? valor = null;

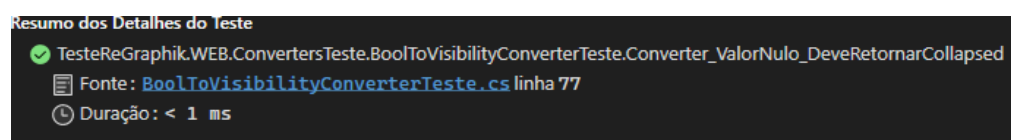
    // Act
    var resultado = converter.Convert(
        valor!,
        typeof(Visibility),
        null!,
        CultureInfo.InvariantCulture);

    // Assert
    Assert.Equal(Visibility.Collapsed, resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT005 – Conversão inversa não implementada

Camada / pasta	WEB / ConvertersTeste
Classe de teste	BoolToVisibilityConverterTeste
Nome do método	ConverterDeVolta_QuandoExecutado_DeveLancarNotImplementedException
Situação	Aprovado

Objetivo: Confirmar o comportamento definido para o método ConvertBack.

Procedimento previsto: O método ConvertBack será chamado com um valor de Visibility para verificar a exceção prevista na implementação.

Resultado esperado: O teste deverá capturar NotImplementedException.

Figura 10 – Evidência do código do CT005 – BoolToVisibilityConverterTeste

```

/// <summary> Confere se a conversão inversa lança a exceção esperada, já que es ...
[Fact]
0 referências
public void ConverterDeVolta_QuandoExecutado_DeveLancarExcecao()
{
    // Arrange
    var converter = new BoolToVisibilityConverter();

    // Act
    Action acao = () => converter.ConvertBack(
        Visibility.Visible,
        typeof(bool),
        null!,
        CultureInfo.InvariantCulture);

    // Assert
    Assert.Throws<NotImplementedException>(acao);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

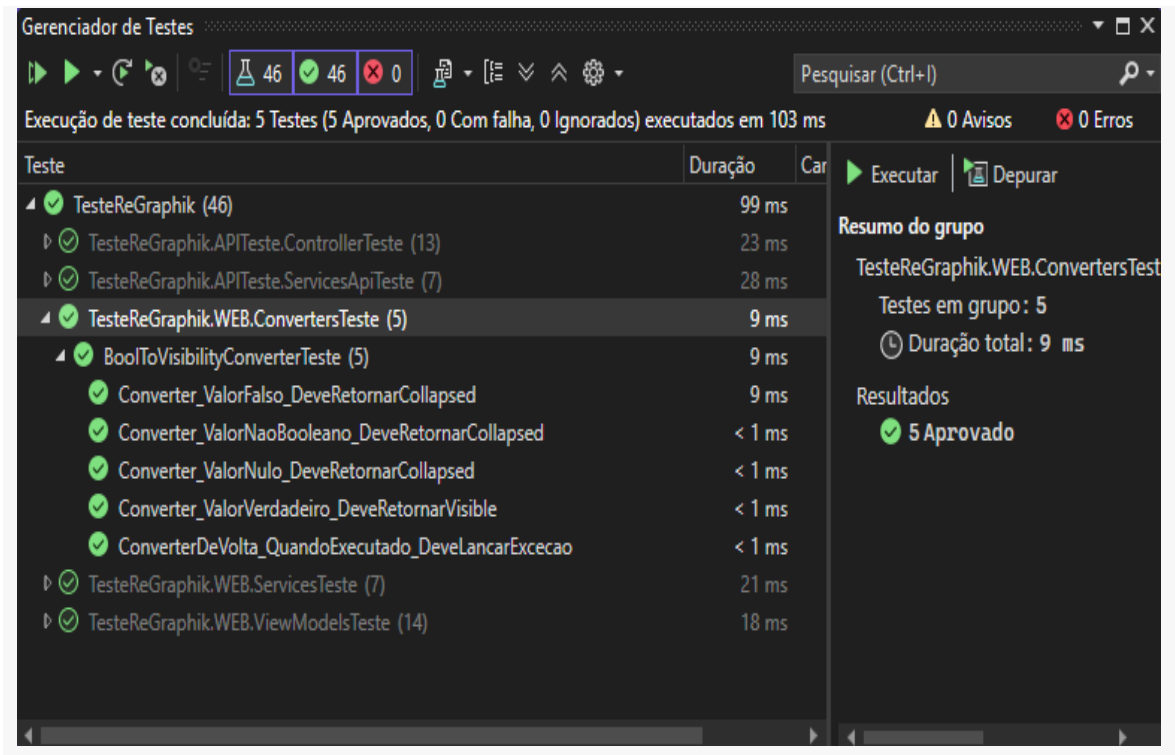
```

Resumo dos Detalhes do Teste
✓ TesteReGraphik.WEB.ConvertersTeste.BoolToVisibilityConverterTeste.ConverterDeVolta_QuandoExecutado_DeveLancarExcecao
  Fonte: BoolToVisibilityConverterTeste.cs linha 99
  Duração: < 1 ms

```

EVIDÊNCIA GERAL DA PASTA – WEB / ConvertersTeste

Figura 11 – Resultado geral dos testes – WEB / ConvertersTeste no Gerenciador de Testes



Fonte: Elaborado pelo autor (2026).

Resumo após execução da pasta: Total: [5] Aprovados: [5] Falhas: [0]
Ignorados: [0].

5.2 WEB / ServicesTeste

Os casos CT006 a CT012 avaliarão o processamento das respostas do Google Places com comunicação HTTP controlada pela suíte.

CT006 – Resposta válida da API

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_RespostaValida_DeveRetornarPontoComDadosCorretos
Situação	Aprovado

Objetivo: Verificar se uma resposta válida do Google Places é transformada corretamente em um ponto de coleta.

Procedimento previsto: Será fornecido um JSON controlado contendo place_id, nome, endereço e coordenadas. A chamada HTTP será simulada, sem acesso real à internet.

Resultado esperado: O teste deverá encontrar um único ponto e validar ID, nome, endereço, material e coordenadas.

Figura 12 – Evidência do código do CT006 – GooglePlacesServiceTeste

```

/// <summary> Confere se uma resposta válida da API é transformada corretamente ...
[Fact]
public async Task BuscarPostos_RespostaValida_DeveRetornarPontoComDadosCorretos()
{
    // Arrange
    string json = """
    {
        "results": [
            {
                "place_id": "local-001",
                "name": "Ponto de Reciclagem Central",
                "formatted_address": "Centro, Belo Horizonte - MG",
                "geometry": {
                    "location": {
                        "lat": -19.9167,
                        "lng": -43.9345
                    }
                }
            }
        ]
    }
    """;

    var servico = CriarServico(json);

    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Belo Horizonte",
        "Plástico");

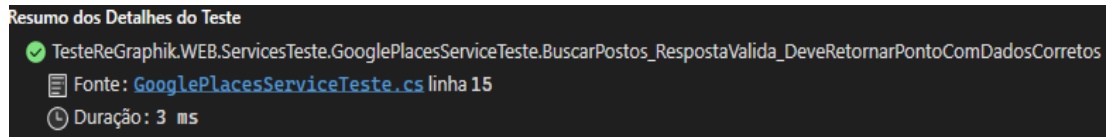
    // Assert
    var ponto = Assert.Single(resultado);

    Assert.Equal("local-001", ponto.Id);
    Assert.Equal("Ponto de Reciclagem Central", ponto.NomePonto);
    Assert.Equal("Centro, Belo Horizonte - MG", ponto.Cidade);
    Assert.Equal("Plástico", ponto.ResiduosAceitos);
    Assert.Equal(-19.9167, ponto.Lat, 4);
    Assert.Equal(-43.9345, ponto.Lng, 4);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT007 – Pesquisa sem resultados

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_SemResultados_DeveRetornarListaVazia
Situação	Aprovado

Objetivo: Verificar o retorno quando a API informa uma lista vazia de resultados.

Procedimento previsto: Será simulado um JSON com a propriedade results contendo um vetor vazio.

Resultado esperado: O teste deverá confirmar que a lista retornada está vazia.

Figura 13 – Evidência do código do CT007 – GooglePlacesServiceTeste

```

/// <summary> Verifica se uma pesquisa sem resultados retorna uma lista vazia.
[Fact]
| 0 referências
public async Task BuscarPostos_SemResultados_DeveRetornarListaVazia()
{
    // Arrange
    string json = ""
    {
        "results": []
    }
    "";

    var servico = CriarServico(json);

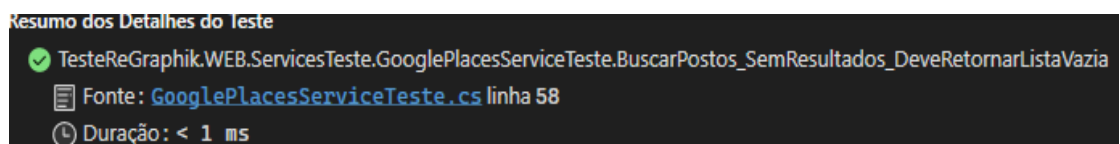
    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Belo Horizonte",
        "Vidro");

    // Assert
    Assert.Empty(resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT008 – Falha na comunicação com a API

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_FalhaNaRequisicao_DeveRetornarListaVazia
Situação	Aprovado

Objetivo: Verificar o tratamento de uma falha HTTP durante a consulta externa.

Procedimento previsto: O manipulador HTTP de teste lançará `HttpRequestException` no momento da requisição.

Resultado esperado: O teste deverá confirmar que a falha é tratada e que o método retorna uma lista vazia.

Figura 14 – Evidência do código do CT008 – GooglePlacesServiceTeste

```

/// <summary> Verifica se uma falha de comunicação é tratada sem interromper o f ..
[Fact]
0 referências
public async Task BuscarPostos_FalhaNaRequisicao_DeveRetornarListaVazia()
{
    // Arrange
    var manipulador = new HttpResponseMessageHandlerFalso(_ =>
        throw new HttpRequestException("Falha simulada"));

    var httpClient = new HttpClient(manipulador);

    var servico = new GooglePlacesService(
        httpClient,
        "chave-teste");

    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Belo Horizonte",
        "Papel");

    // Assert
    Assert.Empty(resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

Resumo dos Detalhes do teste

- ✓ TesteReGraphik.WEB.ServicesTeste.GooglePlacesServiceTeste.BuscarPostos_FalhaNaRequisicao_DeveRetornarListaVazia
- Fonte: `GooglePlacesServiceTeste.cs` linha 83
- Duração: < 1 ms

CT009 – Resultado sem place_id

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_SemPlaceId_DeveGerarIdSequencial
Situação	Aprovado

Objetivo: Verificar a regra usada quando a resposta não possui place_id.

Procedimento previsto: Será fornecido um resultado válido sem o campo place_id.

Resultado esperado: O teste deverá confirmar que o serviço gera o identificador alternativo previsto.

Figura 15 – Evidência do código do CT009 – GooglePlacesServiceTeste

```

/// <summary> Confere se um resultado sem identificador recebe um ID gerado pelo ...
[Fact]
public async Task BuscarPostos_SemPlaceId_DeveGerarIdSequencial()
{
    // Arrange
    string json = """
    {
        "results": [
            {
                "name": "Ponto sem identificador",
                "geometry": {
                    "location": {
                        "lat": -19.90,
                        "lng": -43.90
                    }
                }
            }
        ]
    }
    """;

    var servico = CriarServico(json);

    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Belo Horizonte",
        "Metal");

    // Assert
    var ponto = Assert.Single(resultado);

    Assert.Equal("1", ponto.Id);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

Resumo dos Detalhes do Teste
✓ TesteReGraphik.WEB.ServicesTeste.GooglePlacesServiceTeste.BuscarPostos_SemPlaceId_DeveGerarIdSequencial
  Fonte: GooglePlacesServiceTeste.cs linha 109
  Duração: < 1 ms

```

CT010 – Resultado sem nome

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_SemNome_DeveUsarSemNome
Situação	Aprovado

Objetivo: Verificar o valor adotado quando o Google não informa o nome do local.

Procedimento previsto: Será simulado um resultado com identificador e coordenadas, mas sem a propriedade name.

Resultado esperado: O teste deverá confirmar que NomePonto recebe o texto padrão "Sem Nome".

Figura 16 – Evidência do código do CT010 – GooglePlacesServiceTeste

```

/// <summary> Verifica se um local sem nome recebe a descrição padrão utilizada ...
[Fact]
0 referências
public async Task BuscarPostos_SemNome_DeveUsarSemNome()
{
    // Arrange
    string json = """
    {
        "results": [
            {
                "place_id": "local-002",
                "geometry": {
                    "location": {
                        "lat": -19.90,
                        "lng": -43.90
                    }
                }
            }
        ]
    }
    """;

    var servico = CriarServico(json);

    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Nova Lima",
        "Papel");

    // Assert
    var ponto = Assert.Single(resultado);

    Assert.Equal("Sem Nome", ponto.NomePonto);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

Resumo dos Detalhes do Teste
✓ TesteReGraphik.WEB.ServicesTeste.GooglePlacesServiceTeste.BuscarPostos_SemNome_DeveUsarSemNome
Fonte: GooglePlacesServiceTeste.cs linha 146
Duração: < 1 ms

```

CT011 – Resultado sem coordenadas

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_SemCoordenadas_DeveManterLatitudeELongitudeZero
Situação	Aprovado

Objetivo: Verificar como o serviço trata um resultado sem geometry/location.

Procedimento previsto: Será fornecido um resultado sem latitude e longitude.

Resultado esperado: O teste deverá confirmar que Lat e Lng permanecem com valor 0.

Figura 17 – Evidência do código do CT011 – GooglePlacesServiceTeste

```

/// <summary> Confere o comportamento quando a API retorna um local sem informaç ...
[Fact]
0 referências
public async Task BuscarPostos_SemCoordenadas_DeveManterLatitudeELongitudeZero()
{
    // Arrange
    string json = ""
    {
        "results": [
            {
                "place_id": "local-003",
                "name": "Ponto sem coordenadas"
            }
        ]
    }
    """;

    var servico = CriarServico(json);

    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Nova Lima",
        "Eletrônico");

    // Assert
    var ponto = Assert.Single(resultado);

    Assert.Equal(0.0, ponto.Lat);
    Assert.Equal(0.0, ponto.Lng);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

Resumo dos Detalhes do Teste

- ✓ TesteReGraphik.WEB.ServicesTeste.GooglePlacesServiceTeste.BuscarPostos_SemCoordenadas_DeveManterLatitudeELongitudeZero
- Fonte: [GooglePlacesServiceTeste.cs](#) linha 183
- ⌚ Duração: 5 ms

CT012 – JSON inválido

Camada / pasta	WEB / ServicesTeste
Classe de teste	GooglePlacesServiceTeste
Nome do método	BuscarPostos_JsonInvalido_DeveRetornarListaVazia
Situação	Aprovado

Objetivo: Verificar o tratamento de conteúdo fora do formato JSON esperado.

Procedimento previsto: A resposta HTTP simulada conterá texto inválido em vez de JSON.

Resultado esperado: O teste deverá confirmar que a falha de processamento é tratada e retorna uma lista vazia.

Figura 18 – Evidência do código do CT012 – GooglePlacesServiceTeste

```

/// <summary> Verifica se uma resposta em formato inválido é tratada sem causar ...
[Fact]
✓ | 0 referências
public async Task BuscarPostos_JsonInvalido_DeveRetornarListaVazia()
{
    // Arrange
    string jsonInvalido = ""
    isto nao e um json valido
    "";

    var servico = CriarServico(jsonInvalido);

    // Act
    var resultado = await servico.BuscarPostosNoBrasilAsync(
        "Belo Horizonte",
        "Plástico");

    // Assert
    Assert.Empty(resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

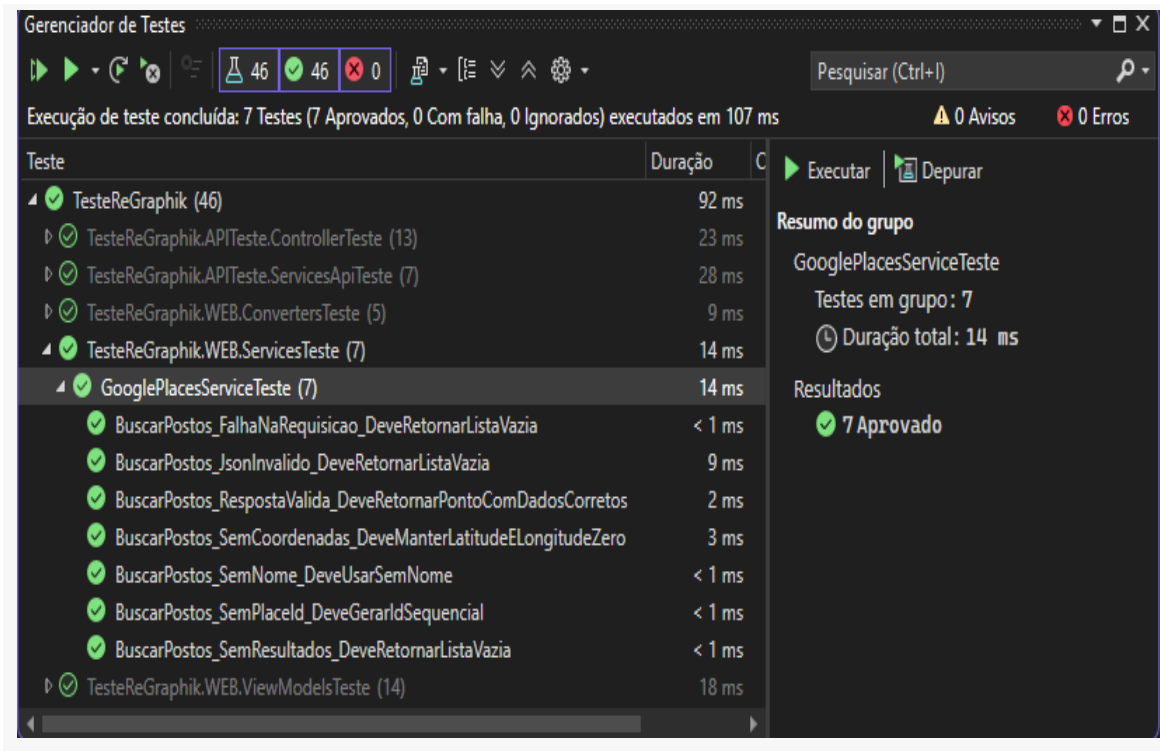
```

Resumo dos Detalhes do Teste
✓ TesteReGraphik.WEB.ServicesTeste.GooglePlacesServiceTeste.BuscarPostos_JsonInvalido_DeveRetornarListaVazia
  Fonte: GooglePlacesServiceTeste.cs linha 216
  Duração: 9 ms

```

EVIDÊNCIA GERAL DA PASTA – WEB / ServicesTeste

Figura 19 – Resultado geral dos testes – WEB / ServicesTeste no Gerenciador de Testes



Fonte: Elaborado pelo autor (2026).

Resumo após execução da pasta: Total: [7] Aprovados: [7] Falhas: [0] Ignorados: [0].

5.3 WEB / ViewModelsTeste

Os casos CT013 a CT026 avaliarão estado de tela, notificações de binding, geração de marcadores e foco no mapa.

CT013 – Inicialização do mapa

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	Construtor_AoCriarViewModel_DeveInicializarMapaLivre
Situação	Aprovado

Objetivo: Verificar o estado inicial do MapaViewModel.

Procedimento previsto: Uma nova instância do ViewModel será criada sem realizar consultas externas.

Resultado esperado: O teste deverá confirmar a existência do comando de busca, do HTML básico do mapa e do estado vazio inicial.

Figura 20 – Evidência do código do CT013 – MapaViewModelTeste

```
public class MapaViewModelTeste
{
    /// <summary> Confere se o mapa é preparado corretamente quando o ViewModel é cr ...
    [Fact]
    0 referências
    public void Construtor_AoCriarViewModel_DeveInicializarMapaLivre()
    {
        // Arrange e Act
        var viewModel = new MapaViewModel();

        // Assert
        Assert.NotNull(viewModel.BuscarCommand);
        Assert.NotEmpty(viewModel.HtmlMapa);
        Assert.Contains("<div id='map'></div>", viewModel.HtmlMapa);
        Assert.True(viewModel.MostrarEstadoVazio);
    }
}
```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```
✓ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.Construtor_AoCriarViewModel_DeveInicializarMapaLivre
  Fonte: MapaViewModelTeste.cs linha 14
  Duração: < 1 ms
```

CT014 – Estado vazio sem pontos

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	MostrarEstadoVazio_SemPontosENaoCarregando_DeveRetornarVerdadeiro
Situação	Aprovado

Objetivo: Verificar se o estado vazio é exibido quando não existem pontos e não há carregamento.

Procedimento previsto: A coleção PontosAtuais será mantida vazia e IsCarregando será definido como false.

Resultado esperado: MostrarEstadoVazio deverá retornar verdadeiro.

Figura 21 – Evidência do código do CT014 – MapaViewModelTeste

```

/// <summary> Verifica se o estado vazio é exibido quando ainda não existem pont

[Fact]
0 referências
public void MostrarEstadoVazio_SemPontosENaoCarregando_DeveRetornarVerdadeiro()
{
    // Arrange
    var viewModel = new MapaViewModel();

    viewModel.PontosAtuais = new ObservableCollection<PontosColeta>();
    viewModel.IsCarregando = false;

    // Act
    var resultado = viewModel.MostrarEstadoVazio;

    // Assert
    Assert.True(resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.MostrarEstadoVazio_SemPontosENaoCarregando_DeveRetornarVerdadeiro
  Fonte: MapaViewModelTeste.cs linha 31
  Duração: < 1 ms

```

CT015 – Estado vazio com ponto

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	MostrarEstadoVazio_ComPonto_DeveRetornarFalso
Situação	Aprovado

Objetivo: Verificar se o estado vazio é ocultado quando existe um ponto de coleta.

Procedimento previsto: Será adicionada uma instância de PontosColeta à coleção e o carregamento permanecerá inativo.

Resultado esperado: MostrarEstadoVazio deverá retornar falso.

Figura 22 – Evidência do código do CT015 – MapaViewModelTeste

```

/// <summary> Confere se o estado vazio deixa de aparecer quando existe pelo men .
[Fact]
0 referências
public void MostrarEstadoVazio_ComPonto_DeveRetornarFalso()
{
    // Arrange
    var viewModel = new MapaViewModel();

    viewModel.PontosAtuais = new ObservableCollection<PontosColeta>
    {
        CriarPonto()
    };

    viewModel.IsCarregando = false;

    // Act
    var resultado = viewModel.MostrarEstadoVazio;

    // Assert
    Assert.False(resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.MostrarEstadoVazio_ComPonto_DeveRetornarFalso
  Fonte: MapaViewModelTeste.cs linha 51
  Duração: < 1 ms

```

CT016 – Estado durante carregamento

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	MostrarEstadoVazio_DuranteCarregamento_DeveRetornarFalso
Situação	Aprovado

Objetivo: Verificar o estado da interface durante uma operação de carregamento.

Procedimento previsto: A coleção permanecerá vazia, porém IsCarregando será definido como true.

Resultado esperado: MostrarEstadoVazio deverá retornar falso enquanto o carregamento estiver ativo.

Figura 23 – Evidência do código do CT016 – MapaViewModelTeste

```

/// <summary> Verifica se a mensagem de estado vazio não aparece enquanto uma b

[Fact]
0 referências
public void MostrarEstadoVazio_DuranteCarregamento_DeveRetornarFalso()
{
    // Arrange
    var viewModel = new MapaViewModel
    {
        PontosAtuais = new ObservableCollection<PontosColeta>(),
        IsCarregando = true
    };

    // Act
    var resultado = viewModel.MostrarEstadoVazio;

    // Assert
    Assert.False(resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.MostrarEstadoVazio_DuranteCarregamento_DeveRetornarFalso
  Fonte: MapaViewModelTeste.cs linha 75
  Duração: 56 ms

```

CT017 – Notificação da propriedade Cidade

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	Cidade_AoAlterarValor_DeveNotificarMudanca
Situação	Aprovado

Objetivo: Verificar se a alteração da cidade dispara PropertyChanged.

Procedimento previsto: O teste assinará o evento PropertyChanged e, em seguida, alterará a propriedade Cidade.

Resultado esperado: O nome da propriedade notificada deverá ser Cidade.

Figura 24 – Evidência do código do CT017 – MapaViewModelTeste

```

/// <summary> Confere se a alteração da cidade é comunicada para a interface at

[Fact]
0 referências
public void Cidade_AoAlterarValor_DeveNotificarMudanca()
{
    // Arrange
    var viewModel = new MapaViewModel();
    string? propriedadeAlterada = null;

    viewModel.PropertyChanged += (_, e) =>
    {
        propriedadeAlterada = e.PropertyName;
    };

    // Act
    viewModel.Cidade = "Nova Lima";

    // Assert
    Assert.Equal(
        nameof(MapaViewModel.Cidade),
        propriedadeAlterada);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.Cidade_AoAlterarValor_DeveNotificarMudanca
  Fonte: MapaViewModelTeste.cs linha 96
  Duração: < 1 ms

```

CT018 – Notificação da coleção PontosAtuais

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	PontosAtuais_AoAlterarColecao_DeveNotificarMudancas
Situação	Aprovado

Objetivo: Verificar as notificações produzidas quando a coleção de pontos é substituída.

Procedimento previsto: O evento PropertyChanged será monitorado durante a atribuição de uma nova ObservableCollection.

Resultado esperado: Deverão ser notificadas PontosAtuais e MostrarEstadoVazio.

Figura 25 – Evidência do código do CT018 – MapaViewModelTeste

```

/// <summary> Verifica se a troca da coleção de pontos informa a mudança para a
[Fact]
0 referências
public void PontosAtuais_AoAlterarColecao_DeveNotificarMudancas()
{
    // Arrange
    var viewModel = new MapaViewModel();
    var propriedadesAlteradas = new List<string?>();

    viewModel.PropertyChanged += (_, e) =>
    {
        propriedadesAlteradas.Add(e.PropertyName);
    };

    // Act
    viewModel.PontosAtuais =
        new ObservableCollection<PontosColeta>();

    // Assert
    Assert.Contains(
        nameof(MapaViewModel.PontosAtuais),
        propriedadesAlteradas);

    Assert.Contains(
        nameof(MapaViewModel.MostrarEstadoVazio),
        propriedadesAlteradas);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.PontosAtuais_AoAlterarColecao_DeveNotificarMudancas
  Fonte: MapaViewModelTeste.cs linha 121
  Duração: 1 ms

```


CT019 – Lista nula de marcadores

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_ListaNula_DeveRetornarArrayVazio
Situação	Aprovado

Objetivo: Verificar se uma referência nula é tratada de forma segura pela geração de marcadores.

Procedimento previsto: GerarJsonMarcadores será chamado com lista nula.

Resultado esperado: O retorno deverá ser o vetor JSON vazio [] sem lançamento de exceção.

Figura 26 – Evidência do código do CT019 – MapaViewModelTeste

```

/// <summary> Confere se uma lista nula é tratada sem causar erro durante a ge

[Fact]
0 referências
public void GerarJsonMarcadores_ListaNula_DeveRetornarArrayVazio()
{
    // Arrange
    var viewModel = new MapaViewModel();

    // Act
    var resultado = viewModel.GerarJsonMarcadores(null!);

    // Assert
    Assert.Equal("[]", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_ListaNula_DeveRetornarArrayVazio
  Fonte: MapaViewModelTeste.cs linha 151
  Duração: < 1 ms

```

CT020 – Lista vazia de marcadores

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_ListaVazia_DeveRetornarArrayVazio
Situação	Aprovado

Objetivo: Verificar o JSON produzido quando não existem pontos de coleta.

Procedimento previsto: Será enviada uma lista vazia para GerarJsonMarcadores.

Resultado esperado: O retorno deverá ser exatamente [].

Figura 27 – Evidência do código do CT020 – MapaViewModelTeste

```

/// <summary> Verifica o resultado quando nenhuma informação de ponto de coleta

[Fact]
0 referências
public void GerarJsonMarcadores_ListaVazia_DeveRetornarArrayVazio()
{
    // Arrange
    var viewModel = new MapaViewModel();
    var pontos = new List<PontosColeta>();

    // Act
    var resultado = viewModel.GerarJsonMarcadores(pontos);

    // Assert
    Assert.Equal("[]", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_ListaVazia_DeveRetornarArrayVazio
  Fonte: MapaViewModelTeste.cs linha 168
  Duração: < 1 ms

```

CT021 – Geração de marcador válido

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_ComPontoValido_DeveGerarDadosCorretos
Situação	Aprovado

Objetivo: Verificar a estrutura gerada para um ponto de coleta válido.

Procedimento previsto: Será criado um ponto com nome, cidade, resíduos e coordenadas conhecidas e o JSON produzido será inspecionado.

Resultado esperado: O JSON deverá conter índice, nome, endereço, tipos de resíduos, latitude e longitude corretos.

Figura 28 – Evidência do código do CT021 – MapaViewModelTeste

```

/// <summary> Confere se os dados de um ponto válido são incluídos corretamente
[Fact]
0 referências
public void GerarJsonMarcadores_ComPontoValido_DeveGerarDadosCorretos()
{
    // Arrange
    var viewModel = new MapaViewModel();

    var pontos = new List<PontosColeta>
    {
        new PontosColeta
        {
            Id = "1",
            NomePonto = "Ponto Central",
            Cidade = "Belo Horizonte",
            Estado = "MG",
            CEP = "30100-000",
            ResiduosAceitos = "Papel e Plástico",
            Lat = -19.9167,
            Lng = -43.9345
        }
    };

    // Act
    var resultado = viewModel.GerarJsonMarcadores(pontos);

    // Assert
    Assert.Contains("\"idx\": 0", resultado);
    Assert.Contains("\"nome\": \"Ponto Central\"", resultado);
    Assert.Contains("\"endereco\": \"Belo Horizonte\"", resultado);
    Assert.Contains("\"tipos\": \"Papel e Plástico\"", resultado);
    Assert.Contains("\"lat\": -19.9167", resultado);
    Assert.Contains("\"lng\": -43.9345", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_ComPontoValido_DeveGerarDadosCorretos
  Fonte: MapaViewModelTeste.cs linha 186
  Duração: 11 ms

```

CT022 – Índices de vários marcadores

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_ComVariosPontos_DeveGerarIndicesSequenciais
Situação	Aprovado

Objetivo: Verificar a numeração dos marcadores quando existem vários pontos.

Procedimento previsto: Serão fornecidos três pontos em uma lista ordenada.

Resultado esperado: O JSON deverá conter índices 0, 1 e 2 na mesma ordem dos pontos informados.

Figura 29 – Evidência do código do CT022 – MapaViewModelTeste

```

/// <summary> Verifica se vários pontos recebem índices diferentes seguindo a or
[Fact]
0 referências
public void GerarJsonMarcadores_ComVariosPontos_DeveGerarIndicesSequenciais()
{
    // Arrange
    var viewModel = new MapaViewModel();

    var pontos = new List<PontosColeta>
    {
        CriarPonto("Ponto 1"),
        CriarPonto("Ponto 2"),
        CriarPonto("Ponto 3")
    };

    // Act
    var resultado = viewModel.GerarJsonMarcadores(pontos);

    // Assert
    Assert.Contains("\"idx\": 0", resultado);
    Assert.Contains("\"idx\": 1", resultado);
    Assert.Contains("\"idx\": 2", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_ComVariosPontos_DeveGerarIndicesSequenciais
  Fonte: MapaViewModelTeste.cs linha 223
  Duração: < 1 ms

```

CT023 – Coordenadas iguais a zero

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_CoordenadasZero_DeveUsarCoordenadasPadrao
Situação	Aprovado

Objetivo: Verificar a regra de coordenadas padrão quando o ponto possui latitude e longitude zero.

Procedimento previsto: Será enviado um ponto com Lat = 0 e Lng = 0.

Resultado esperado: O JSON deverá utilizar as coordenadas padrão definidas pelo ViewModel.

Figura 30 – Evidência do código do CT023 – MapaViewModelTeste

```

/// <summary> Confere se as coordenadas padrão são utilizadas quando latitude e longitude são zero
[/Fact]
0 referências
public void GerarJsonMarcadores_CoordenadasZero_DeveUsarCoordenadasPadrao()
{
    // Arrange
    var viewModel = new MapaViewModel();

    var pontos = new List<PontosColeta>
    {
        new PontosColeta
        {
            NomePonto = "Ponto sem coordenadas",
            Cidade = "São Paulo",
            ResiduosAceitos = "Papel",
            Lat = 0,
            Lng = 0
        }
    };

    // Act
    var resultado = viewModel.GerarJsonMarcadores(pontos);

    // Assert
    Assert.Contains("\"lat\": -23.55052", resultado);
    Assert.Contains("\"lng\": -46.633308", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_CoordenadasZero_DeveUsarCoordenadasPadrao
  Fonte: MapaViewModelTeste.cs linha 249
  Duração: < 1 ms

```

CT024 – Formatação decimal das coordenadas

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_CoordenadasDecimais_DeveUsarPontoComoSeparador
Situação	Aprovado

Objetivo: Verificar se a serialização das coordenadas é independente da cultura regional.

Procedimento previsto: Será utilizado um ponto com várias casas decimais.

Resultado esperado: O JSON deverá usar ponto como separador decimal e não vírgula.

Figura 31 – Evidência do código do CT024 – MapaViewModelTeste

```

/// <summary> Verifica se coordenadas decimais são geradas usando ponto como sep
[Fact]
0 referências
public void GerarJsonMarcadores_CoordenadasDecimais_DeveUsarPontoComoSeparador()
{
    // Arrange
    var viewModel = new MapaViewModel();

    var pontos = new List<PontosColeta>
    {
        new PontosColeta
        {
            NomePonto = "Ponto Teste",
            Cidade = "Nova Lima",
            ResiduosAceitos = "Vidro",
            Lat = -19.923456,
            Lng = -43.934567
        }
    };

    // Act
    var resultado = viewModel.GerarJsonMarcadores(pontos);

    // Assert
    Assert.Contains("-19.923456", resultado);
    Assert.Contains("-43.934567", resultado);

    Assert.DoesNotContain("-19,923456", resultado);
    Assert.DoesNotContain("-43,934567", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_CoordenadasDecimais_DeveUsarPontoComoSeparador
  Fonte: MapaViewModelTeste.cs linha 279
  Duração: < 1 ms

```

CT025 – Tratamento de caracteres especiais

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	GerarJsonMarcadores_TextoComCaracteresEspeciais_DeveEscaparConteudo
Situação	Aprovado

Objetivo: Verificar o tratamento de aspas, barras e quebras de linha nos textos enviados ao mapa.

Procedimento previsto: Será criado um ponto contendo caracteres especiais em nome, cidade e resíduos.

Resultado esperado: O conteúdo deverá ser escapado ou normalizado de forma a não quebrar o JSON/JavaScript.

Figura 32 – Evidência do código do CT025 – MapaViewModelTeste

```

/// <summary> Confere se caracteres que poderiam interferir no JavaScript são tr

[Fact]
0 referências
public void GerarJsonMarcadores_TextoComCaracteresEspeciais_DeveEscaparConteudo()
{
    // Arrange
    var viewModel = new MapaViewModel();

    var pontos = new List<PontosColeta>
    {
        new PontosColeta
        {
            NomePonto = "Ponto \"Central\"",
            Cidade = "Nova Lima\nCentro",
            ResiduosAceitos = @"Papel\Vidro",
            Lat = -19.98,
            Lng = -43.85
        }
    };

    // Act
    var resultado = viewModel.GerarJsonMarcadores(pontos);

    // Assert
    Assert.Contains("Ponto \\\"Central\\\"", resultado);
    Assert.Contains("Nova Lima\nCentro", resultado);
    Assert.Contains("Papel\\\"Vidro\"", resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.GerarJsonMarcadores_TextoComCaracteresEspeciais_DeveEscaparConteudo
  Fonte: MapaViewModelTeste.cs linha 312
  Duração: < 1 ms

```

CT026 – Foco em ponto do mapa

Camada / pasta	WEB / ViewModelsTeste
Classe de teste	MapaViewModelTeste
Nome do método	FocarNoPonto_MapaCarregado_DeveDefinirIndiceCorreto
Situação	Aprovado

Objetivo: Verificar o índice de foco após o carregamento do mapa.

Procedimento previsto: Serão adicionados dois pontos à coleção; o mapa será marcado como carregado e o segundo ponto será solicitado para foco.

Resultado esperado: IndiceFoco deverá corresponder à posição do segundo ponto na coleção.

Figura 33 – Evidência do código do CT026 – MapaViewModelTeste

```

/// <summary> Verifica se o mapa identifica corretamente o ponto que deve recebe
[Fact]
0 referências
public void FocarNoPonto_MapaCarregado_DeveDefinirIndiceCorreto()
{
    // Arrange
    var viewModel = new MapaViewModel();

    var primeiroPonto = CriarPonto("Ponto 1");
    var segundoPonto = CriarPonto("Ponto 2");

    viewModel.PontosAtuais =
        new ObservableCollection<PontosColeta>
        {
            primeiroPonto,
            segundoPonto
        };

    viewModel.NotificarMapaCarregado();

    // Act
    viewModel.FocarNoPonto(segundoPonto);

    // Assert
    Assert.Equal(1, viewModel.IndiceFoco);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

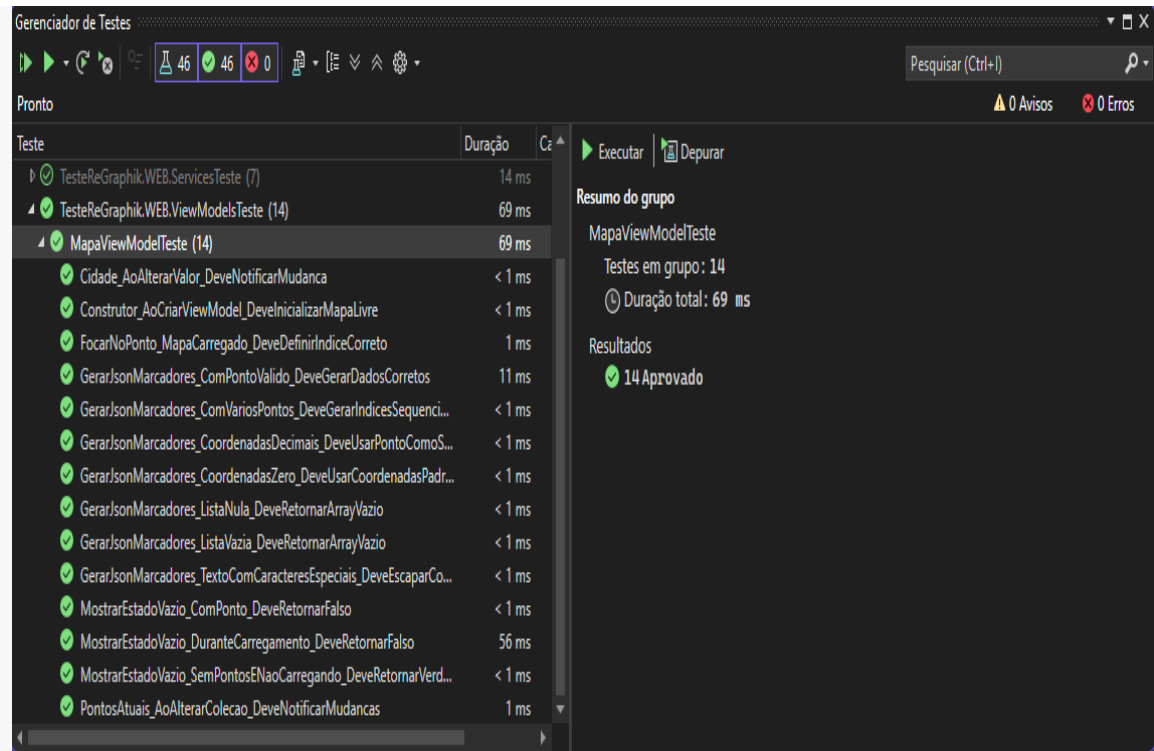
```

✓ TesteReGraphik.WEB.ViewModelsTeste.MapaViewModelTeste.FocarNoPonto_MapaCarregado_DeveDefinirIndiceCorreto
  Fonte: MapaViewModelTeste.cs linha 343
  Duração: 1 ms

```


EVIDÊNCIA GERAL DA PASTA – WEB / ViewModelsTeste

Figura 34 – Resultado geral dos testes – WEB / ViewModelsTeste no Gerenciador de Testes



Fonte: Elaborado pelo autor (2026).

Resumo após execução da pasta: Total: [14] Aprovados: [14] Falhas: [0] Ignorados: [0].

6 PLANO DE TESTES UNITÁRIOS – API

Esta seção reunirá os testes das classes da ApiRestReGraphik relacionadas aos pontos de coleta. As dependências de Firebase e Google serão substituídas por respostas controladas quando o objetivo for avaliar apenas a regra da classe sob teste.

6.1 APITeste / ControllerTeste

Os casos CT027 a CT039 avaliarão as respostas HTTP da Controller para listagem, sincronização, consulta, cadastro, atualização e exclusão.

CT027 – Listagem de pontos com sucesso

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Listar_ComDados_DeveRetornarOk
Situação	Aprovado

Objetivo: Verificar a resposta do endpoint GET quando o serviço fornece pontos cadastrados.

Procedimento previsto: O serviço controlado retornará uma lista com um ponto, e o método Get será executado sem acesso real ao Firebase.

Resultado esperado: O Controller deverá retornar 200 OK e uma coleção de PontosColetaDto com os dados esperados.

Figura 35 – Evidência do código do CT027 – PontosColetaControllerTeste

```

/// <summary> Confere se a listagem retorna os pontos cadastrados quando existem ...
[Fact]
0 referências
public async Task Listar_ComDados_DeveRetornarOk()
{
    // Arrange
    var servico = new PontosColetaServiceFalso();

    servico.PontosListados.Add(new PontosColeta
    {
        Id = "ponto-01",
        NomePonto = "Ponto Central",
        Cidade = "Nova Lima",
        Estado = "MG",
        CEP = "34000-000",
        ResiduosAceitos = "Papel e Plástico",
        Lat = -19.985,
        Lng = -43.846
    });

    var controller = CriarController(servico);

    // Act
    var resultado = await controller.Get();

    // Assert
    var respostaOk =
        Assert.IsType<OkObjectResult>(resultado);

    var pontos =
        Assert.IsAssignableFrom<IEnumerable<PontosColetaDto>>(
            respostaOk.Value);

    var ponto = Assert.Single(pontos);

    Assert.Equal("ponto-01", ponto.Id);
    Assert.Equal("Ponto Central", ponto.NomePonto);
    Assert.Equal("Nova Lima", ponto.Cidade);
    Assert.Equal("MG", ponto.Estado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.Listar_ComDados_DeveRetornarOk
  Fonte: PontosDeColetaControllerTeste.cs linha 21
  Duração: 3 ms

```

CT028 – Falha de comunicação na listagem

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Listar_FalhaComunicacao_DeveRetornarBadGateway
Situação	Aprovado

Objetivo: Verificar a resposta HTTP quando a listagem sofre `HttpRequestException`.

Procedimento previsto: O serviço falso lançará uma falha de comunicação durante Listar.

Resultado esperado: O Controller deverá retornar 502 Bad Gateway.

Figura 36 – Evidência do código do CT028 – PontosColetaControllerTeste

```

/// <summary> Verifica se uma falha de comunicação durante a listagem retorna o
[Fact]
0 referências
public async Task Listar_FalhaComunicacao_DeveRetornarBadGateway()
{
    // Arrange
    var servico = new PontosColetaServiceFalso
    {
        ExcecaoListar =
            new HttpRequestException("Falha simulada")
    };

    var controller = CriarController(servico);

    // Act
    var resultado = await controller.Get();

    // Assert
    var resposta =
        Assert.IsType<ObjectResult>(resultado);

    Assert.Equal(
        StatusCodes.Status502BadGateway,
        resposta.StatusCode);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.Listar_FalhaComunicacao_DeveRetornarBadGateway
  Fonte: PontosDeColetaControllerTeste.cs linha 64
  Duração: 3 ms

```

CT029 – Sincronização sem cidade

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	SincronizarCidade_CidadeVazia_DeveRetornarBadRequest
Situação	Aprovado

Objetivo: Verificar a validação do parâmetro cidade na sincronização.

Procedimento previsto: SincronizarCidade será chamado com texto vazio.

Resultado esperado: O Controller deverá retornar 400 Bad Request sem chamar a sincronização externa.

Figura 37 – Evidência do código do CT029 – PontosColetaControllerTeste

```

/// <summary> Confere se a sincronização é recusada quando a cidade não é inform ...
[Fact]
0 referências
public async Task SincronizarCidade_CidadeVazia_DeveRetornarBadRequest()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso();

    var controller =
        CriarController(servico);

    // Act
    var resultado =
        await controller.SincronizarCidade("");

    // Assert
    Assert.IsType<BadRequestObjectResult>(
        resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.SincronizarCidade_CidadeVazia_DeveRetornarBadRequest
Fonte: PontosDeColetaControllerTeste.cs linha 92
Duração: < 1 ms

```

CT030 – Sincronização sem chave da API

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	SincronizarCidade_SemApiKey_DeveRetornarErroServidor
Situação	Aprovado

Objetivo: Verificar a resposta quando GoogleMaps:ApiKey não está configurada.

Procedimento previsto: A Controller será criada com uma IConfiguration sem a chave do Google Maps.

Resultado esperado: O retorno deverá ser 500 Internal Server Error.

Figura 38 – Evidência do código do CT030 – PontosColetaControllerTeste

```

/// <summary> Verifica o comportamento quando a chave do Google Maps não está co ...
[Fact]
0 referências
public async Task SincronizarCidade_SemApiKey_DeveRetornarErroServidor()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso();

    var configuracao =
        CriarConfiguracaoSemApiKey();

    var controller =
        CriarController(
            servico,
            configuracao);

    // Act
    var resultado =
        await controller.SincronizarCidade(
            "Nova Lima");

    // Assert
    var resposta =
        Assert.IsType<ObjectResult>(resultado);

    Assert.Equal(
        StatusCodes.Status500InternalServerError,
        resposta.StatusCode);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.API.Teste.ControllerTeste.PontosColetaControllerTeste.SincronizarCidade_SemApiKey_DeveRetornarErroServidor
  Fonte: PontosDeColetaControllerTeste.cs linha 115
  Duração: 85 ms

```

CT031 – Sincronização concluída com sucesso

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	SincronizarCidade_Valida_DeveRetornarOk
Situação	Aprovado

Objetivo: Verificar a resposta da sincronização para uma cidade válida.

Procedimento previsto: O serviço falso informará três pontos salvos e um ignorado.

Resultado esperado: O Controller deverá retornar 200 OK com Mensagem, PontosSalvos = 3 e PontosIgnoradosPorDuplicidade = 1.

Figura 39 – Evidência do código do CT031 – PontosColetaControllerTeste

```

/// <summary> Confere se uma sincronização válida retorna as quantidades de pont ...
[Fact]
0 referências
public async Task SincronizarCidade_Valida_DeveRetornarOk()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso
        {
            ResultadoSincronizacao = (3, 1)
        };

    var controller =
        CriarController(servico);

    // Act
    var resultado =
        await controller.SincronizarCidade(
            "Nova Lima");

    // Assert
    var respostaOk =
        Assert.IsType<OkObjectResult>(
            resultado);

    Assert.NotNull(respostaOk.Value);

    var pontosSalvos =
        LerPropriedade<int>(
            respostaOk.Value!,
            "PontosSalvos");

    var pontosIgnorados =
        LerPropriedade<int>(
            respostaOk.Value!,
            "PontosIgnoradosPorDuplicidade");

    var mensagem =
        LerPropriedade<string>(
            respostaOk.Value!,
            "Mensagem");

    Assert.Equal(3, pontosSalvos);
    Assert.Equal(1, pontosIgnorados);
    Assert.Contains("Nova Lima", mensagem);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.SincronizarCidade_Valida_DeveRetornarOk
📄 Fonte: [PontosDeColetaControllerTeste.cs](#) linha 148
🕒 Duração: 7 ms

CT032 – Busca por ID existente

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	BuscarPorId_Existente_DeveRetornarOk
Situação	Aprovado

Objetivo: Verificar a consulta individual de um ponto existente.

Procedimento previsto: O serviço controlado retornará um objeto PontosColeta para o ID informado.

Resultado esperado: O Controller deverá retornar 200 OK com o objeto esperado.

Figura 40 – Evidência do código do CT032 – PontosColetaControllerTeste

```

/// <summary> Confere se um ponto existente pode ser encontrado pelo seu identif ..
[Fact]
| 0 referências
public async Task BuscarPorId_Existente_DeveRetornarOk()
{
    // Arrange
    var ponto =
        CriarPonto();

    var servico =
        new PontosColetaServiceFalso
        {
            PontoPorId = ponto
        };

    var controller =
        CriarController(servico);

    // Act
    var resultado =
        await controller.GetById(
            ponto.Id!);

    // Assert
    var respostaOk =
        Assert.IsType<OkObjectResult>(
            resultado);

    var pontoRetornado =
        Assert.IsType<PontosColeta>(
            respostaOk.Value);

    Assert.Equal(
        ponto.Id,
        pontoRetornado.Id);

    Assert.Equal(
        ponto.NomePonto,
        pontoRetornado.NomePonto);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.BuscarPorId_Existente_DeveRetornarOk
  Fonte: PontosDeColetaControllerTeste.cs linha 197
  Duração: 1 ms

```


CT033 – Busca por ID inexistente

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	BuscarPorId_Inexistente_DeveRetornarNotFound
Situação	Aprovado

Objetivo: Verificar a resposta para um identificador que não existe.

Procedimento previsto: O serviço falso retornará null para ObterPorId.

Resultado esperado: O Controller deverá retornar 404 Not Found.

Figura 41 – Evidência do código do CT033 – PontosColetaControllerTeste

```

/// <summary> Verifica se a busca retorna NotFound quando o ponto não existe.
[Fact]
public async Task BuscarPorId_Inexistente_DeveRetornarNotFound()
{
    // Arrange
    var service =
        new PontosColetaServiceFalso
        {
            PontoPorId = null
        };

    var controller =
        CriarController(service);

    // Act
    var resultado =
        await controller.GetById(
            "id-inexistente");

    // Assert
    Assert.IsType<NotFoundObjectResult>(
        resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.API.Teste.ControllerTeste.PontosColetaControllerTeste.BuscarPorId_Inexistente_DeveRetornarNotFound
  Fonte: PontosDeColetaControllerTeste.cs linha 240
  Duração: < 1 ms

```

CT034 – Cadastro sem dados

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Cadastrar_DtoNulo_DeveRetornarBadRequest
Situação	Aprovado

Objetivo: Verificar a validação de entrada no endpoint POST.

Procedimento previsto: O método Post será chamado com DTO nulo.

Resultado esperado: O Controller deverá retornar 400 Bad Request.

Figura 42 – Evidência do código do CT034 – PontosColetaControllerTeste

```
[Fact]
0 referências
public async Task Cadastrar_DtoNulo_DeveRetornarBadRequest()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso();

    var controller =
        CriarController(servico);

    // Act
    var resultado =
        await controller.Post(null!);

    // Assert
    Assert.IsType<BadRequestObjectResult>(
        resultado);
}
```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```
✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.Cadastrar_DtoNulo_DeveRetornarBadRequest
  Fonte: PontosDeColetaControllerTeste.cs linha 267
  Duração: < 1 ms
```

CT035 – Cadastro válido

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Cadastrar_DadosValidos_DeveRetornarCreated
Situação	Aprovado

Objetivo: Verificar o fluxo de criação de um ponto de coleta válido.

Procedimento previsto: Será fornecido um PontosColetaDto completo e o serviço de criação será controlado pela suíte.

Resultado esperado: O Controller deverá retornar CreatedAtAction/201, gerar um GUID e preservar os dados principais do DTO.

Figura 43 – Evidência do código do CT035 – PontosColetaControllerTeste

```
[Fact]
public async Task Cadastrar_DadosValidos_DeveRetornarCreated()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso();

    var controller =
        CriarController(servico);

    var dto =
        CriarDto();

    // Act
    var resultado =
        await controller.Post(dto);

    // Assert
    var criado =
        Assert.IsType<CreatedAtActionResult>(
            resultado);

    Assert.Equal(
        nameof(PontosColetaController.GetById),
        criado.ActionName);

    var retorno =
        Assert.IsType<PontosColetaDto>(
            criado.Value);

    Assert.Equal(
        dto.NomePonto,
        retorno.NomePonto);

    Assert.Equal(
        dto.Cidade,
        retorno.Cidade);

    Assert.False(
        string.IsNullOrWhiteSpace(
            retorno.Id));

    Assert.True(
        Guid.TryParse(
            retorno.Id,
            out _));

    Assert.NotNull(
        servico.PontoCriado);
}
```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.Cadastrar_DadosValidos_DeveRetornarCreated
📄 Fonte: [PontosDeColetaControllerTeste.cs](#) linha 290
🕒 Duração: 7 ms

CT036 – Atualização de ID inexistente

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Atualizar_IdInexistente_DeveRetornarNotFound
Situação	Aprovado

Objetivo: Verificar a tentativa de atualizar um ponto que não existe.

Procedimento previsto: ObterPorId retornará null antes da atualização.

Resultado esperado: O Controller deverá retornar 404 Not Found e não concluir a atualização.

Figura 44 – Evidência do código do CT036 – PontosColetaControllerTeste

```

/// <summary> Verifica se um ponto inexistente não pode ser atualizado.
[Fact]
public async Task Atualizar_IdInexistente_DeveRetornarNotFound()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso
        {
            PontoPorId = null
        };

    var controller =
        CriarController(servico);

    var dto =
        CriarDto();

    // Act
    var resultado =
        await controller.Put(
            "id-inexistente",
            dto);

    // Assert
    Assert.IsType<NotFoundObjectResult>(
        resultado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.API.Teste.ControllerTeste.PontosColetaControllerTeste.Atualizar_IdInexistente_DeveRetornarNotFound
  Fonte: PontosDeColetaControllerTeste.cs linha 345
  Duração: < 1 ms

```

CT037 – Atualização válida

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Atualizar_DadosValidos_DeveRetornarOk
Situação	Aprovado

Objetivo: Verificar a atualização de um ponto existente.

Procedimento previsto: O serviço falso devolverá um ponto existente; um DTO com novos dados será enviado ao Put.

Resultado esperado: O Controller deverá retornar 200 OK e encaminhar ao serviço o ID e os valores atualizados.

Figura 45 – Evidência do código do CT037 – PontosColetaControllerTeste

```

/// <summary> Confere se os novos dados são aplicados quando o ponto realmente e ..
[Fact]
public async Task Atualizar_DadosValidos_DeveRetornarOk()
{
    // Arrange
    var pontoExistente =
        CriarPonto();

    var servico =
        new PontosColetaServiceFalso
        {
            PontoPorId =
                pontoExistente
        };

    var controller =
        CriarController(servico);

    var dto =
        CriarDto();

    dto.NomePonto =
        "Ponto Atualizado";

    dto.Cidade =
        "Belo Horizonte";

    // Act
    var resultado =
        await controller.Put(
            pontoExistente.Id!,
            dto);

    // Assert
    Assert.IsType<OkObjectResult>(
        resultado);

    Assert.NotNull(
        servico.PontoAtualizado);

    Assert.Equal(
        "Ponto Atualizado",
        servico.PontoAtualizado!
            .NomePonto);

    Assert.Equal(
        "Belo Horizonte",
        servico.PontoAtualizado
            .Cidade);

    Assert.Equal(
        pontoExistente.Id,
        servico.IdAtualizado);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.Atualizar_DadosValidos_DeveRetornarOk
📄 Fonte: [PontosDeColetaControllerTeste.cs](#) linha 376
🕒 Duração: 1 ms

CT038 – Exclusão de ID inexistente

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Excluir_IdInexistente_DeveRetornarNotFound
Situação	Aprovado

Objetivo: Verificar a tentativa de excluir um ponto não encontrado.

Procedimento previsto: ObterPorId retornará null para o identificador informado.

Resultado esperado: O Controller deverá retornar 404 Not Found e não executar a exclusão.

Figura 46 – Evidência do código do CT038 – PontosColetaControllerTeste

```
[Fact]
0 | 0 referências
public async Task Excluir_IdInexistente_DeveRetornarNotFound()
{
    // Arrange
    var servico =
        new PontosColetaServiceFalso
        {
            PontoPorId = null
        };

    var controller =
        CriarController(servico);

    // Act
    var resultado =
        await controller.Delete(
            "id-inexistente");

    // Assert
    Assert.IsType<NotFoundObjectResult>(
        resultado);
}
```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```
✓ TesteReGraphik.APITeste.ControllerTeste.PontosColetaControllerTeste.Excluir_IdInexistente_DeveRetornarNotFound
  Fonte: PontosDeColetaControllerTeste.cs linha 434
  Duração: < 1 ms
```

CT039 – Exclusão válida

Camada / pasta	API / ControllerTeste
Classe de teste	PontosColetaControllerTeste
Nome do método	Excluir_IdExistente_DeveRetornarOk
Situação	Aprovado

Objetivo: Verificar o fluxo de exclusão de um ponto existente.

Procedimento previsto: O serviço falso retornará o ponto solicitado e registrará o ID recebido no método Excluir.

Resultado esperado: O Controller deverá retornar 200 OK e o ID encaminhado à exclusão deverá coincidir com o ponto existente.

Figura 47 – Evidência do código do CT039 – PontosColetaControllerTeste

```
[Fact]
public async Task Excluir_IdExistente_DeveRetornarOk()
{
    // Arrange
    var ponto =
        CriarPonto();

    var servico =
        new PontosColetaServiceFalso
        {
            PontoPorId = ponto
        };

    var controller =
        CriarController(servico);

    // Act
    var resultado =
        await controller.Delete(
            ponto.Id!);

    // Assert
    Assert.IsType<OkObjectResult>(
        resultado);

    Assert.Equal(
        ponto.Id,
        servico.IdExcluido);
}
```

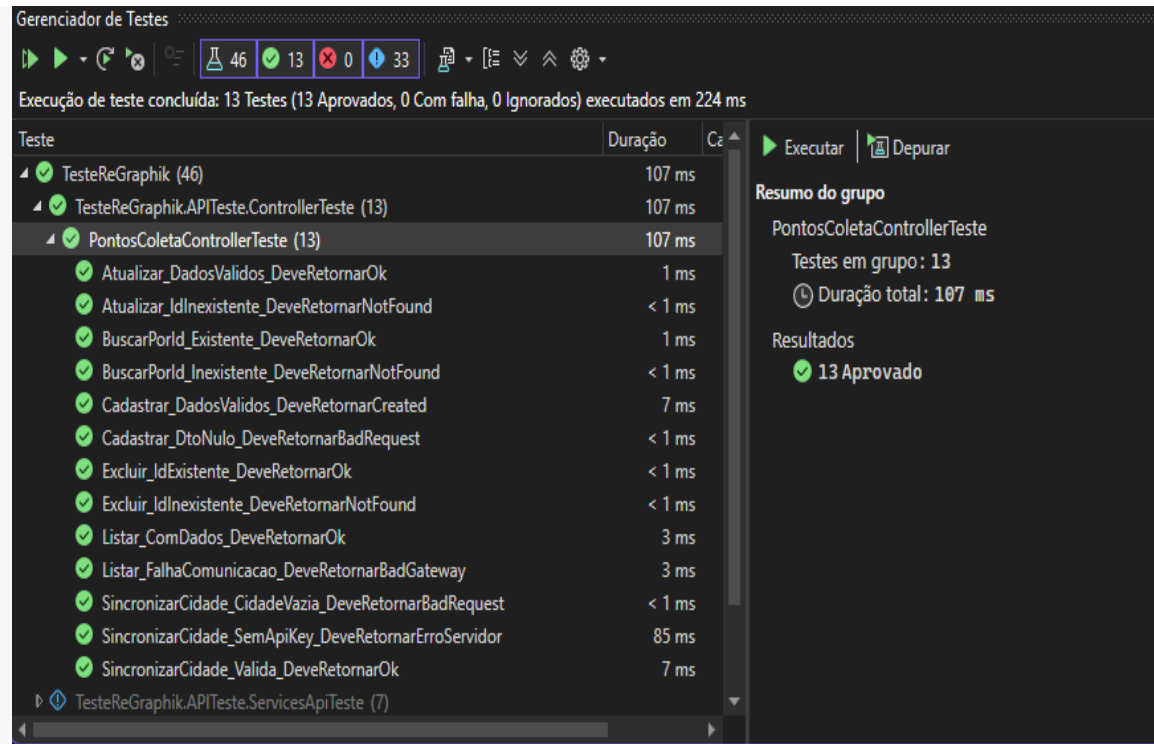
Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```
TesteReGraphik.API.Teste.ControllerTeste.PontosColetaControllerTeste.Excluir_IdExistente_DeveRetornarOk
Fonte: PontosDeColetaControllerTeste.cs linha 461
Duração: < 1 ms
```

EVIDÊNCIA GERAL DA PASTA – APITeste / ControllerTeste

Figura 48 – Resultado geral dos testes – APITeste / ControllerTeste no Gerenciador de Testes



Fonte: Elaborado pelo autor (2026).

Resumo após execução da pasta: Total: [13] Aprovados: [13] Falhas: [0] Ignorados: [0].

6.2 APITeste / ServicesApiTeste

Os casos CT040 a CT046 avaliarão validações de configuração e regras de sincronização do PontosColetaService sem acesso real ao Firebase ou Google.

CT040 – Configuração do Firebase ausente

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	ServicoPontosColeta_SemUrlFirebase_DeveLancarExcecao
Situação	Aprovado

Objetivo: Verificar a validação da URL do Firebase no construtor de produção.

Procedimento previsto: O serviço será instanciado com uma IConfiguration sem Firebase:RealtimeDatabaseUrl.

Resultado esperado: A construção deverá lançar exceção com a mensagem prevista para configuração ausente.

Figura 49 – Evidência do código do CT040 – PontosColetaServiceTeste

```
/// <summary> Confere se o serviço impede a inicialização quando a URL do Fire
[Fact]
public void ServicoPontosColeta_SemUrlFirebase_DeveLancarExcecao()
{
    // Arrange
    var configuracao = new ConfigurationBuilder()
        .AddInMemory(new (variável local) IConfigurationRoot? configuracao)
        .Build();

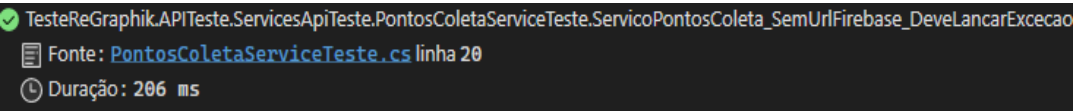
    var logger =
        NullLogger<PontosColetaService>.Instance;

    // Act
    var excecao = Assert.Throws<Exception>(() =>
        new PontosColetaService(
            logger,
            configuracao));

    // Assert
    Assert.Equal(
        "Configurações do Firebase ausentes.",
        excecao.Message);
}
```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:



CT041 – Arquivo de credenciais ausente

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	ServicoPontosColeta_SemArquivoCredencial_DeveLancarFileNotFoundException
Situação	Aprovado

Objetivo: Verificar a validação do arquivo de credenciais do Firebase.

Procedimento previsto: Será configurada uma URL do Firebase e um nome de arquivo inexistente.

Resultado esperado: A construção deverá lançar FileNotFoundException e indicar o arquivo não localizado.

Figura 50 – Evidência do código do CT041 – PontosColetaServiceTeste

```

/// <summary> Verifica se o serviço identifica a ausência do arquivo de credenci ...
[Fact]
public void ServicoPontosColeta_SemArquivoCredencial_DeveLancarFileNotFoundException()
{
    // Arrange
    var valores =
        new Dictionary<string, string?>
        {
            {
                "Firebase:RealtimeDatabaseUrl",
                "https://teste.firebaseio.com/"
            },
            {
                "Firebase:CredentialFilePath",
                "arquivo-que-nao-existe-teste.json"
            }
        };

    var configuracao = new ConfigurationBuilder()
        .AddInMemoryCollection(valores)
        .Build();

    var logger =
        NullLogger<PontosColetaService>.Instance;

    // Act
    var execucao =
        Assert.Throws<FileNotFoundException>(() =>
            new PontosColetaService(
                logger,
                configuracao));

    // Assert
    Assert.Contains(
        "arquivo-que-nao-existe-teste.json",
        execucao.Message);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

TesteReGraphik.APITeste.ServicesApiTeste.PontosColetaServiceTeste.ServicoPontosColeta_SemArquivoCredencial_DeveLancarFileNotFoundException
Fonte: PontosColetaServiceTeste.cs linha 48
Duração: < 1 ms

```

CT042 – Status de erro retornado pelo Google

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	Sincronizar_StatusErroGoogle_DeveRetornarZero
Situação	Aprovado

Objetivo: Verificar a reação da sincronização a um status do Google diferente de OK e ZERO_RESULTS.

Procedimento previsto: A resposta HTTP simulada utilizará REQUEST_DENIED e uma lista vazia.

Resultado esperado: O método deverá retornar zero salvos, zero ignorados e não criar pontos.

Figura 51 – Evidência do código do CT042 – PontosColetaServiceTeste

```

/// <summary> Confere se um status de erro retornado pelo Google encerra a si
[Fact]
public async Task Sincronizar_StatusErroGoogle_DeveRetornarZero()
{
    // Arrange
    var json = """
    {
        "status": "REQUEST_DENIED",
        "results": []
    }
    """;

    var servico =
        CriarServicoControlado();

    var httpClient =
        CriarHttpClient(json);

    // Act
    var resultado =
        await servico.SincronizarComGoogleMapsAsync(
            "Nova Lima",
            "chave-teste",
            httpClient);

    // Assert
    Assert.Equal(0, resultado.salvos);
    Assert.Equal(0, resultado.ignorados);

    Assert.Empty(servico.PontosCriados);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✓ TesteReGraphik.APITeste.ServicesApiTeste.PontosColetaServiceTeste.Sincronizar_StatusErroGoogle_DeveRetornarZero
  Fonte: PontosColetaServiceTeste.cs linha 89
  Duração: < 1 ms

```

CT043 – Resposta sem resultados

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	Sincronizar_SemResults_DeveRetornarZero
Situação	Aprovado

Objetivo: Verificar a sincronização quando a resposta não possui a propriedade results.

Procedimento previsto: Será simulado um JSON com status OK, mas sem results.

Resultado esperado: O método deverá encerrar com contadores zerados e sem criar pontos.

Figura 52 – Evidência do código do CT043 – PontosColetaServiceTeste

```

/// <summary> Verifica o comportamento quando a resposta não possui uma lista de ...
[Fact]
public async Task Sincronizar_SemResults_DeveRetornarZero()
{
    // Arrange
    var json = """
    {
        "status": "OK"
    }
    """;

    var servico =
        CriarServicoControlado();

    var httpClient =
        CriarHttpClient(json);

    // Act
    var resultado =
        await servico.SincronizarComGoogleMapsAsync(
            "Nova Lima",
            "chave-teste",
            httpClient);

    // Assert
    Assert.Equal(0, resultado.salvos);
    Assert.Equal(0, resultado.ignorados);

    Assert.Empty(servico.PontosCriados);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```

✔ TesteReGraphik.APITeste.ServicesApiTeste.PontosColetaServiceTeste.Sincronizar_SemResults_DeveRetornarZero
  Fonte: PontosColetaServiceTeste.cs linha 124
  Duração: < 1 ms

```


CT044 – Ponto duplicado por coordenadas

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	Sincronizar_CoordenadasDuplicadas_DeveIgnorarPonto
Situação	Aprovado

Objetivo: Verificar a regra de prevenção de duplicidade pelas coordenadas.

Procedimento previsto: Um ponto existente será carregado em memória e o Google simulado retornará outro ponto com a mesma latitude e longitude.

Resultado esperado: O método deverá contabilizar um ignorado, zero salvos e não chamar o cadastro do ponto duplicado.

Figura 53 – Evidência do código do CT044 – PontosColetaServiceTeste

```

/// <summary> Confere se um ponto com as mesmas coordenadas de outro já existent ...
[Fact]
public async Task Sincronizar_CoordenadasDuplicadas_DeveIgnorarPonto()
{
    // Arrange
    var pontoExistente = new PontosColeta
    {
        Id = "ponto-existente",
        NomePonto = "Ponto já cadastrado",
        Cidade = "Nova Lima",
        Estado = "MG",
        CEP = "34000-000",
        ResiduosAceitos = "Reciclável",
        Lat = -19.985,
        Lng = -43.846
    };

    var servico =
        CriarServicoControlado();

    servico.PontosExistentes.Add(
        pontoExistente);

    var json = """
    {
        "status": "OK",
        "results": [
            {
                "name": "Ponto retornado pelo Google",
                "geometry": {
                    "location": {
                        "lat": -19.985,
                        "lng": -43.846
                    }
                }
            }
        ]
    }
    """;

    var httpClient =
        CriarHttpClient(json);

    // Act
    var resultado =
        await servico.SincronizarComGoogleMapsAsync(
            "Nova Lima",
            "chave-teste",
            httpClient);

    // Assert
    Assert.Equal(0, resultado.salvos);
    Assert.Equal(1, resultado.ignorados);

    Assert.Empty(servico.PontosCriados);
}

```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```
✓ TesteReGraphik.APITeste.ServicesApiTeste.PontosColetaServiceTeste.Sincronizar_CoordenadasDuplicadas_DevelgnorarPonto  
  Fonte: PontosColetaServiceTeste.cs linha 158  
  Duração: 1 ms
```

CT045 – Novo ponto contabilizado como salvo

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	Sincronizar_NovoPonto_DeveContabilizarSalvo
Situação	Aprovado

Objetivo: Verificar o processamento de um ponto ainda inexistente na base.

Procedimento previsto: O Google simulado retornará um local com coordenadas novas e o método Criar será substituído por armazenamento em memória.

Resultado esperado: O método deverá contabilizar um salvo, zero ignorados e preparar o ponto com os valores de configuração definidos para o teste.

Figura 54 – Evidência do código do CT045 – PontosColetaServiceTeste

```
// <summary> verifica se um ponto novo e preparado para cadastro e co
[Fact]
public async Task Sincronizar_NovoPonto_DeveContabilizarSalvo()
{
    // Arrange
    var servico =
        CriarServicoControlado();

    var json = """
    {
        "status": "OK",
        "results": [
            {
                "name": "Novo Ponto de Coleta",
                "geometry": {
                    "location": {
                        "lat": -19.9901,
                        "lng": -43.8502
                    }
                }
            }
        ]
    }
    """;

    var httpClient =
        CriarHttpClient(json);

    // Act
    var resultado =
        await servico.SincronizarComGoogleMapsAsync(
            "Nova Lima",
            "chave-teste",
            httpClient);

    // Assert
    Assert.Equal(1, resultado.salvos);
    Assert.Equal(0, resultado.ignorados);

    var pontoCriado =
        Assert.Single(servico.PontosCriados);

    Assert.Equal(
        "Novo Ponto de Coleta",
        pontoCriado.NomePonto);

    Assert.Equal(
        "Nova Lima",
        pontoCriado.Cidade);

    Assert.Equal(
        "MG",
        pontoCriado.Estado);

    Assert.Equal(
        "34000-000",
        pontoCriado.CEP);

    Assert.Equal(
        "Reciclável",
        pontoCriado.ResiduosAceitos);

    Assert.Equal(
        -19.9901,
        pontoCriado.Lat,
        4);

    Assert.Equal(
        -43.8502,
        pontoCriado.Lng,
        4);
}
```

Fonte: Elaborado pelo autor (2026).

Resultado após execução:

✓ TesteReGraphik.APITeste.ServicesApiTeste.PontosColetaServiceTeste.Sincronizar_NovoPonto_DeveContabilizarSalvo
📄 Fonte: [PontosColetaServiceTeste.cs](#) linha 218
🕒 Duração: 35 ms

CT046 – JSON inválido na sincronização

Camada / pasta	API / ServicesApiTeste
Classe de teste	PontosColetaServiceTeste
Nome do método	Sincronizar_JsonInvalido_DeveLancarInvalidOperationException
Situação	Aprovado

Objetivo: Verificar o tratamento de uma resposta externa corrompida.

Procedimento previsto: O HttpClient de teste retornará texto que não pode ser interpretado como JSON.

Resultado esperado: A sincronização deverá converter a falha de desserialização para InvalidOperationException, conforme o tratamento previsto no serviço.

Figura 55 – Evidência do código do CT046 – PontosColetaServiceTeste

```
[Fact]
public async Task Sincronizar_JsonInvalido_DeveLancarInvalidOperationException()
{
    // Arrange
    var servico =
        CriarServicoControlado();

    var httpClient =
        CriarHttpClient(
            "isso nao e um json valido");

    // Act
    var execucao =
        await Assert.ThrowsAsync<InvalidOperationException>(
            async () =>
                await servico.SincronizarComGoogleMapsAsync(
                    "Nova Lima",
                    "chave-teste",
                    httpClient));

    // Assert
    Assert.Contains(
        "corrompidos",
        execucao.Message);
}
```

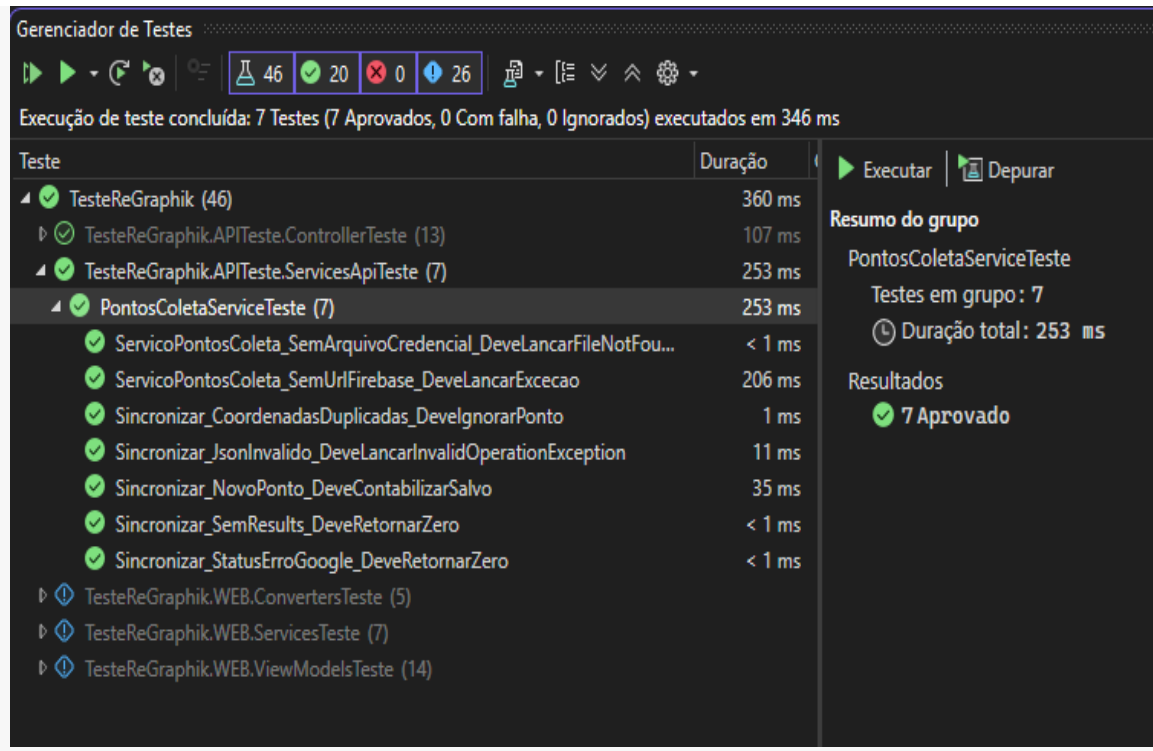
Fonte: Elaborado pelo autor (2026).

Resultado após execução:

```
TesteReGraphik.APITeste.ServicesApiTeste.PontosColetaServiceTeste.Sincronizar_JsonInvalido_DeveLancarInvalidOperationException
Fonte: PontosColetaServiceTeste.cs linha 294
Duração: 11 ms
```

EVIDÊNCIA GERAL DA PASTA – APITeste / ServicesApiTeste

Figura 56 – Resultado geral dos testes – APITeste / ServicesApiTeste no Gerenciador de Testes



Fonte: Elaborado pelo autor (2026).

Resumo após execução da pasta: Total: [7] Aprovados: [7] Falhas: [0] Ignorados: [0].

7 REGISTRO DOS RESULTADOS DA SUÍTE

Total planejado	46
Total executado	46
Aprovados	46
Falhas encontradas	0
Falhas corrigidas	0
Resultado final	46 Testes aprovados.

8 CONSIDERAÇÕES FINAIS

A realização dos testes unitários no módulo de Mapa e Pontos de Coleta do sistema ReGraphik permitiu verificar de forma organizada os principais comportamentos das camadas WEB e API relacionadas a essa funcionalidade. Foram elaborados e executados 46 casos de teste, contemplando diferentes cenários, como entradas válidas e inválidas, tratamento de valores nulos, respostas inesperadas, falhas de comunicação, validações, geração de marcadores, sincronização com serviços externos, prevenção de duplicidade, operações de cadastro, consulta, atualização e exclusão.

Ao final da execução da suíte, os 46 testes foram aprovados, sem ocorrência de falhas. Esse resultado demonstra que os comportamentos avaliados apresentaram as respostas esperadas dentro dos cenários definidos para a atividade.

Durante a preparação da suíte, também foi necessário realizar pequenas adaptações de testabilidade em classes relacionadas ao módulo, principalmente para permitir o isolamento de dependências externas como Google Places e Firebase. Essas adaptações não alteraram as regras de negócio nem o funcionamento normal da aplicação, servindo apenas para possibilitar a execução de testes de forma controlada, repetível e independente de conexões externas.

A atividade contribuiu para validar o funcionamento do módulo e também para demonstrar a importância dos testes unitários no desenvolvimento de software. A utilização do xUnit, juntamente com a organização dos testes no padrão Arrange, Act e Assert, facilitou a leitura dos cenários, a identificação dos resultados esperados e a manutenção da suíte.

Dessa forma, a suíte desenvolvida cumpriu o objetivo proposto, finalizando a validação do módulo de Mapa e Pontos de Coleta com **46 testes executados, 46 aprovados e nenhuma falha**, fornecendo maior segurança para a continuidade e manutenção do sistema ReGraphik.

REFERÊNCIAS

SENAI. Aula 02 – Criando o primeiro projeto xUnit.net. Material didático do curso Técnico em Desenvolvimento de Sistemas. Nova Lima, 2026.

SENAI. Aula 03 – Escrevendo os primeiros testes com xUnit.net. Material didático do curso Técnico em Desenvolvimento de Sistemas. Nova Lima, 2026.

SENAI. Situação de Aprendizagem – Suíte Individual de Testes Unitários Aplicada ao Projeto de TCC. Nova Lima, 2026.

REGRAPHIK. Código-fonte do módulo Mapa e Pontos de Coleta. Projeto de TCC, 2026.